

6

Adding Video and Editing Functionality to Packtagram

Having already mastered the art of capturing stunning photographs and applying mesmerizing filters with CameraX, it's time to elevate our Packtagram app to new heights. Now, we will embark on an exciting new venture: diving into the world of video.

Videos are not just moving pictures; they are powerful storytelling tools that breathe life into our apps. They create dynamic interactions, keeping users engaged and offering them a canvas to express creativity. In this chapter, we'll guide you through the process of integrating video capabilities into your app, akin to adding a new dimension to the Instagram-like experience we have been crafting.

We will start by exploring how to capture high-quality videos using the CameraX library, an extension of the skills you've already honed for photo capture. Then, we'll delve into the world of **Fast Forward Moving Picture Expert Group (Ffmpeg)**, a robust library for video processing, to add layers of creativity to your videos – from simple captions that convey messages to sophisticated filters that transform the visual mood.

You'll learn to not only capture and edit videos but also to efficiently upload them to Firebase Storage, ensuring that your app can handle large files seamlessly and provide a smooth user experience.

By the end of this chapter, you will have added a significant feature to your app, making it not just a photo-sharing platform but a comprehensive multimedia experience.

To accomplish that, in this chapter, we will cover the following topics:

- Adding video functionality to our app
- Getting to know FFMpeg
- Adding a caption to a video with FFMpeg
- Adding a filter to a video with FFMpeg
- Uploading the video

Technical requirements

As in the previous chapter, you will need to have installed Android Studio (or another editor of your preference).

You can find the complete code that we will be using in this chapter in this book's GitHub repository:

<https://github.com/PacktPublishing/Thriving-in-Android-Development-using-Kotlin/tree/main/Chapter-6>.

Adding video functionality to our app

In this section, we will extend the functionality of our Android app so that it includes video-capturing capabilities through CameraX. This powerful library not only simplifies the process of capturing photos but also provides an efficient way to record videos. We'll start by adapting our existing CameraX setup, which is designed for capturing photos, to also handle video recording. The aim is to provide a seamless integration, maintaining the simplicity and robustness of CameraX.

First, we need to set up the preview for the video recording. In the previous chapter, we created a **CameraPreview** composable. We'll reuse the same composable here:

```
@Composable
fun CameraPreview(cameraController:
LifecycleCameraController, modifier: Modifier = Modifier) {
    AndroidView(
        factory = { context ->
            PreviewView(context).apply {
                implementationMode =
                    PreviewView.ImplementationMode.COMPATIBLE
            }
        },
        modifier = modifier,
        update = { previewView ->
            previewView.controller = cameraController
        }
    )
}
```

Now, we need to create a new button composable to record images and sound from the preview (instead of just capturing the image):

```
@Composable
fun CaptureVideoButton(
    cameraController: LifecycleCameraController,
    onRecordingFinished: (String) -> Unit,
) {
    val context = LocalContext.current
    val recording = remember {
        mutableStateOf<Recording?>(null) }
    IconButton(
        onClick = {
            cameraController.setEnabledUseCases(
                LifecycleCameraController.VIDEO_CAPTURE
            )
            if (recording.value == null) {
                recording.value =
```

```

        startRecording(cameraController,
            context, onRecordingFinished)
    } else {
        stopRecording(recording.value)
        recording.value = null
    }
},
modifier = Modifier
    .size(60.dp)
    .padding(8.dp),
) {
    Icon(
        painter = if (recording.value == null)
            painterResource(id =
                R.drawable.ic_videocam) else
            painterResource(id =
                R.drawable.ic_stop),
        contentDescription = "Capture video",
        tint = MaterialTheme.colorScheme.onPrimary
    )
}
}

```

Here, we are creating a new composable called **CaptureVideoButton**. It is similar to the **CaptureButton** composable but with some modifications. For example, now, we'll need to create a variable **recording**. The **Recording** class in CameraX is responsible for managing an active video recording session. It encapsulates the state and operations needed to start, pause, resume, and stop the recording. In our code, the **recording** variable will be used to manage the current recording session.

Once the user clicks the button, we'll configure the video capture use case,

```
cameraController.setEnabledUseCases(LifecycleCameraController.VIDEO_CAPTURE),
```

so that **cameraController** can start and manage the video recording process, ensuring that the camera is correctly set up for capturing high-quality video and enabling the necessary configurations and resources for the recording session to proceed smoothly. Then, if a recording hasn't been already initiated, we'll start a new recording. If it has already been initiated, we'll stop it.

The icon of the button will show a camera prior to the recording being initiated and a stop button if the recording is already in progress, to indicate to the user that they should click it to stop the recording.

To finish this recording functionality, we need to implement the **startRecording** function:

```

@SuppressLint("MissingPermission")
private fun startRecording(
    cameraController: LifecycleCameraController,
    context: Context,
    onRecordingFinished: (String) -> Unit
): Recording {
    val videoFile = File(context.filesDir,
        "video_${System.currentTimeMillis()}.mp4")

```

```

        val outputOptions =
            FileOutputOptions.Builder(videoFile).build()
        val audioConfig = AudioConfig.create(true)
        val executor = Executors.newSingleThreadExecutor()
        return cameraController.startRecording(
            outputOptions,
            audioConfig,
            executor
        ) { recordEvent ->
            when (recordEvent) {
                is VideoRecordEvent.Finalize -> {
                    if (recordEvent.hasError()) {
                        Log.e("CaptureVideoButton",
                            "Video recording error:
                                ${recordEvent.error}")
                    } else {
                        onRecordingFinished(
                            videoFile.absolutePath)
                    }
                }
            }
        }
    }
}

```

This function is marked with the `@SuppressLint("MissingPermission")` annotation, indicating an assumption that the necessary runtime permissions, such as access to the camera and microphone, have already been granted. We will handle these permissions the same way we did with the photo capture, so the annotation is safe to use here as the permissions would have already been granted.

The function begins by defining the location and filename for the video recording. It uses the `File` class to create a reference to a `video_${System.currentTimeMillis()}.mp4` file, which is stored in the app-specific directory on the external storage. This approach to file storage is advantageous as it does not require additional permissions and ensures that the stored data is private to the application.

Next, the code sets up `FileOutputOptions` using the previously defined file. This step is crucial as it configures how the recorded video data will be written to the filesystem. The `FileOutputOptions` class, part of the CameraX library, offers an intuitive API to set these parameters efficiently – for example, it allows us to specify the video location using `ContentResolver` (you can find additional information about `FileOutputOptions` here:

<https://developer.android.com/reference/androidx/camera/video/FileOutputOptions>). Next, the audio configuration is created, in this case to allow audio using `AudioConfig.create(true)`.

Then, an executor is created using `Executors.newSingleThreadExecutor()`, which facilitates the execution of tasks in a background thread, thereby keeping the UI thread unblocked and responsive. With these parameters defined (`fileOutputOptions`, `AudioConfig`, and `Executor`), we can execute the `cameraController.startRecording` function, which will initiate the recording.

Additionally, an event listener is defined using the **Consumer<VideoRecordEvent>** interface. This listener uses a **when** statement to handle different types of **VideoRecordEvent**, such as **VideoRecordEvent.Finalize**, which indicates the completion of the recording. The event listener also checks for errors during the recording process, ensuring robust error handling.

Then, a **Recording** object is returned, representing the ongoing recording session. This recording object is crucial for the next step.

Now, let's implement the **stopRecording** function:

```
fun stopRecording(recording: Recording?) {
    recording?.stop()
}
```

In this concise and straightforward function, we only have one line of code, but it does something essential. The function takes a single parameter, **recording**, which is our instance of the **Recording** class from the CameraX library.

The core action in this function is to invoke **stop()** on the **recording** object. When this method is called, it tells the **recording** instance to terminate the current video recording session. This involves stopping video frames from being captured and finalizing the video file that's being recorded. The video file is then saved to the location that's specified when the recording has finished.

Now, we will include the new button in **CaptureModeContent**, which we built previously for the capture feature:

```
@Composable
private fun CaptureModeContent(
    cameraController: LifecycleCameraController,
    onImageCaptured: (Bitmap) -> Any,
    onVideoCaptured: (String) -> Any
) {
    Box(modifier = Modifier.fillMaxSize()) {
        CameraPermissionRequester {
            Box(
                contentAlignment = Alignment.BottomCenter,
                modifier = Modifier.fillMaxSize()
            ) {
                CameraPreview(...)
                Row {
                    CaptureButton(...)
                    CaptureVideoButton(
                        cameraController =
                            cameraController,
                        onRecordingFinished = { videoPath ->
                            onVideoCaptured(videoPath)
                        }
                    )
                }
            }
        }
    }
}
```

```
    }  
    }  
    }  
}
```

Here, we have added a **Row** composable to show both buttons horizontally side by side. We have also added a new Lambda (**onVideoCaptured**) that we will use to pass the video file path when the recording has finished.

With these changes, we should be able to see the newly implemented button:



Figure 6.1: The video capture button already integrated into the StoryContent screen when the video is not being recorded

When we click the video capture button, we should see its icon change to the stop symbol:



Figure 6.2: Video recording in progress

And with this, we are ready to record our videos using CameraX! Now, it is time for us to learn how to modify or edit the recorded videos. With these aspects in mind, let me introduce you to the FFmpeg library.

Getting to know FFmpeg

FFmpeg is an open-source multimedia framework that has become a cornerstone in the world of audio and video processing. Renowned for its versatility and power, FFmpeg offers a comprehensive suite of libraries and tools to handle video, audio, and other multimedia files and streams. At its core, FFmpeg is a command-line tool, enabling users to convert media files from one format into another, manipulate video and audio recordings, and perform a wide array of other multimedia processing tasks.

NOTE

You can find the official FFmpeg documentation here: <https://ffmpeg.org/>.

Through the following subsections, we will learn what components are part of FFmpeg, its key features, and how to integrate this powerful library in our Android apps.

The components of FFmpeg

The FFmpeg project is composed of several components, each serving a specific role in multimedia processing:

- **libavcodec:** A library containing decoders and encoders for audio/video codecs
- **libavformat:** This library deals with the container formats, managing the multiplexing and demultiplexing aspects of multimedia streams
- **libavutil:** A utility library that provides a range of helper functions and data structures
- **libavfilter:** Used for applying various audio and video filters
- **libswscale:** Dedicated to handling image scaling and color format conversions

Together, these components provide a robust foundation for handling a wide array of multimedia processing tasks.

Key features of FFmpeg

FFmpeg stands out for its extensive range of capabilities. Some of its key features include the following:

- **Format support:** FFmpeg supports a vast number of audio and video formats, both in terms of encoding and decoding, making it incredibly versatile for multimedia processing
- **Conversion:** It can convert media files between various formats with high efficiency, a feature that's widely used in various applications and services

- **Streaming:** FFmpeg excels in streaming capabilities, allowing for audio and video to be captured, encoded, and streamed in real time
- **Filtering:** With its powerful filtering capabilities, users can apply various transformations, overlays, and effects to their media

Integrating mobile-ffmpeg into our project

In the context of Android development, FFmpeg can be used as a powerful tool for video editing functionalities, such as applying filters, transcoding, or even adding subtitles. However, integrating FFmpeg into Android applications while using C++ code requires using **Java Native Interface (JNI)**. Luckily, there is an easier option, which is leveraging an Android-compatible wrapper of FFmpeg, such as **mobile-ffmpeg**.

mobile-ffmpeg is a specialized port of FFmpeg that's designed for mobile platforms such as Android and iOS. It provides pre-built binaries, mobile-specific APIs, and optimizations tailored to the constraints of mobile hardware. This makes it easier to integrate FFmpeg's powerful capabilities into mobile applications, allowing developers to leverage advanced multimedia processing features with less complexity.

To integrate the **mobile-ffmpeg** library into our project, we will start by opening our **libs.versions.toml** file. There, we will add the version and the library group and name:

```
[versions]
...
mobileffmpeg = "4.4"
[libraries]
...
mobileffmpeg = { group = "com.arthenica", name = "mobile-ffmpeg-full", version.ref = "mobileffmpeg"
```

Here, we have just added the latest **mobile-ffmpeg** version and the library reference to our version catalog.

As always, to use it in any of our modules, we will have to add the dependency in the **build.gradle.kts** file:

```
dependencies {
    ....
    implementation(libs.mobileffmpeg)
}
```

Once it's added to our dependencies, we will have to sync our Gradle files so that it's ready to be used in our code. But first, let's learn how FFmpeg works and can be used.

Understanding the FFmpeg command-line syntax

As we have seen, FFmpeg is a powerful multimedia framework that's capable of decoding, encoding, transcoding, multiplexing (joining, for example, audio and video in a single file), demultiplexing (separating audio

and video in different files), streaming, filtering, and playing almost any type of media file. Understanding its command-line syntax is crucial for effective video processing, especially in Android environments.

Keep in mind that we will not be executing these commands in a terminal, but the **mobile-ffmpeg** library uses the same syntax to allow us to execute them using a function called **FFmpeg.execute()**, as we will see now.

At its core, an FFmpeg command follows a basic structure:

```
FFmpeg.execute("[global_options] {[input_file_options] [flags] input_url} ... {[output_file_option
```

Let's take a closer look at the components of this syntax:

- **global_options**: These are settings that can be applied throughout the command, such as configuring logging levels or overriding default configurations.
- **input_file_options**: These are options that specifically affect the input file, such as the format, codec, or frame rate.
- **input_url**: The path to the input file.
- **output_file_options**: These are similar to input file options but they affect the output file, such as the format, codec, or bitrate.
- **output_url**: The path for the output file.
- **options/flags**: These start with a dash (-) and modify how FFmpeg processes files. The most used options and flags are as follows:
 - **-i**: Specifies the input file
 - **-c**: Indicates the codec; use **-c:v** for video and **-c:a** for audio
 - **-b**: Sets the bitrate; **-b:v** for video and **-b:a** for audio
 - **-s**: Defines the frame size (resolution)
 - **-r**: Sets the frame rate
 - **-f**: Indicates the format

Let's see how we can use this syntax to complete some basic operations.

Basic conversion

Converting a video file from one format into another is a fundamental task in video editing. For example, converting an MP4 file into an AVI file can be done like so:

```
FFmpeg.execute("-i input.mp4 output.avi")
```

This command tells FFmpeg to take **input.mp4** and convert it into **output.avi** using the default settings for codecs and quality (the default values are used here because we didn't specify any settings).

Specifying codecs

Codecs are algorithms that are used for encoding (compressing) or decoding (decompressing) video and audio streams. In FFmpeg, you can specify different codecs for the video and audio components of a file:

- **Video codec:** A video codec processes the visual data in the file. Choosing the right video codec affects the video's quality, size, and compatibility with different players and devices.
- **Audio codec:** An audio codec deals with the sound component. It determines the audio quality, file size, and compatibility with audio playback systems.

To specify codecs in FFmpeg, use the **-c** flag followed by a colon, then either **v** for video or **a** for audio, and then specify the codec's name:

```
ffmpeg -i input.file -c:v [video_codec] -c:a [audio_codec] output.file
```

So, for example, to specify the H.264 and AAC codecs, you can run the following command:

```
ffmpeg -i input.mp4 -c:v libx264 -c:a aac output.mp4
```

Let's understand what the values of this command mean:

- **-i:** This indicates that the next parameter is going to be the input file.
- **input.mp4:** This is the route to the input file.
- **-c:v libx264:** This value sets the video codec to **libx264**, a popular codec for H.264 video encoding. It's known for its efficiency and compatibility with most video platforms.
- **-c:a aac:** This value sets the audio codec to **aac** (which stands for Advanced Audio Coded), known for good quality audio at lower bitrates, making it ideal for web videos.
- **output.mp4:** This indicates the route to the output file.

Note that higher-quality codecs often result in larger file sizes – the balance between quality and file size can be key, depending on the use case.

Also, it is important to know that some codecs require licensing for commercial use (for example, H.264), whereas others are open source and free (for example, VP9 and Opus).

Adjusting video quality

In video processing, one of the most crucial aspects to manage is the quality of the output video. The quality is often directly influenced by the bitrate. The **bitrate** is measured in **bits per second (bps)** and represents the amount of video or audio data that's encoded for 1 second of playback. Higher bitrates generally mean better quality but also larger file sizes.

There are two types of bitrate:

- **Constant bitrate (CBR):** This encodes the file at a consistent bitrate throughout, leading to predictable file sizes but potentially varying quality
- **Variable bitrate (VBR):** This adjusts the bitrate according to the complexity of each part of the video, balancing quality and file size more

To adjust the bitrate in FFmpeg, we can use the **-b:v** flag for video bitrate and **-b:a** for audio bitrate:

```
ffmpeg -i input.file -b:v [video_bitrate] -b:a [audio_bitrate] output.file
```

For example, to set standard definition video with moderate quality, we can run the following command:

```
ffmpeg -i input.mp4 -b:v 1500k -b:a 128k output.mp4
```

Let's see what the values of this command mean:

- **-i:** This indicates that the next parameter is going to be the input file
- **input.mp4:** This is the route to the input file
- **-b:v 1500k:** Sets the video bitrate to 1,500 kbps, which is suitable for standard-definition content
- **-b:a 128k:** Sets the audio bitrate to 128 kbps, providing decent audio quality without excessive file size
- **output.mp4:** Indicates the route to the output file

It's worth noting that lower bitrates may lead to noticeable compression artifacts, especially in fast-moving or complex scenes. On the other hand, higher bitrates offer better quality but at the expense of larger file sizes, which might be an issue for online streaming or limited storage.

Resizing video

Resizing or scaling videos is a common task in video editing, whether it's to fit different screen sizes, reduce file size, or conform to specific resolution requirements. FFmpeg offers powerful tools to resize videos with ease, but understanding the impact of these changes is crucial for maintaining quality.

But what are video resolution and aspect ratio?

- **Resolution:** The resolution of a video is the dimension in pixels, given as width x height. Standard resolutions include 480p (SD), 720p (HD), 1080p (Full HD), and 4K (Ultra HD).
- **Aspect ratio:** This is the ratio of the width to the height of the video. Common aspect ratios are 16:9 (widescreen) and 4:3 (traditional).

To resize videos in FFmpeg, the **-s** (size) flag is used. It sets the resolution:

```
ffmpeg -i input.file -s [width]x[height] output.file
```

For example, to resize to 1080p, the command will be as follows:

```
ffmpeg -i input.mp4 -s 1920x1080 output.mp4
```

Let's see what the values of this command mean:

- **-i:** Indicates that the next parameter is going to be the input file
- **input.mp4:** The route to the input file
- **-s 1920x1080:** Resizes the video to full HD (1080p), which is suitable for high-quality presentations and large displays
- **output.mp4:** Indicates the route to the output file

There are some things to consider when resizing videos:

- Choose the resolution based on where and how the video will be viewed. For instance, you should choose a high resolution for TV broadcasts and something lower for web or mobile use.
- Higher resolutions lead to larger files, which can be a concern for storage and streaming.
- Always consider the quality of the source video. Upscaling low-quality footage might not yield desirable results.

Now that we are familiar with the basic features of FFmpeg, we will learn about the advanced ones.

Advanced syntax and options in FFmpeg

FFmpeg's true power lies in its advanced options, allowing for sophisticated manipulation and processing of audio and video files. This section delves deeper into these advanced features, providing insights into how they can be leveraged for complex tasks.

Using filters for enhanced video and audio manipulation

FFmpeg comes equipped with an extensive range of filters for both video and audio. These can be applied to tasks such as cropping, rotating, adding watermarks, and adjusting brightness or contrast.

To apply filters, you can use the **-vf** (video filters) or **-af** (audio filters) option. Here is the schema of how the filter syntax would work:

```
ffmpeg -i input.file -vf "[filter1],[filter2]" output.file
```

For example, imagine a scenario where you need to crop a video and adjust its color properties. You can do this by running the following command:

```
ffmpeg -i input.mp4 -vf "crop=640:480:0:0, hue=h=60:s=1" -c:a copy output.mp4
```

Let's take a closer look at the values of this command:

- **-i:** Indicates that the next parameter is going to be the input file.
- **input.mp4:** This is the route to the input file.
- **-vf:** This stands for video filters, and allows you to apply one or more filters to the video stream.

- **crop=640:480:0:0**: This is the crop filter. It crops the video to a width of 640 pixels and a height of 480 pixels. The 0:0 value at the end specifies the x and y coordinates of the top-left corner of the crop area. In this case, it's set to the top-left corner of the original video. So, this filter effectively crops the video to a 640x480 rectangle starting from the top-left corner.
- **hue=h=60:s=1**: There are two parts to this code:
 - **h=60** adjusts the hue of the video. Hue is a color component that allows us to shift colors on a 360-degree color wheel. A value of 60 shifts the colors by 60 degrees. For example, blue might become green, red might become yellow, and so on.
 - **s=1** sets the saturation level. A saturation of 1 means that the colors are left as-is in terms of intensity. Decreasing this value would desaturate the colors, leading to a more grayscale image.
- **-c:a**: Resizes the video to full HD (1080p).
- **output.mp4**: Indicates the route to the output file.

In summary, this FFmpeg command reads **input.mp4**, crops the video to a 640x480 resolution starting from the top-left corner, shifts the hue of the video colors by 60 degrees on the color wheel, maintains the original saturation, copies the audio without re-encoding, and saves all these changes in **output.mp4**.

Using an overlay video filter

The overlay filter in FFmpeg is a versatile feature that allows users to superimpose one video or image over another. This is particularly useful for adding logos, watermarks, subtitles, picture-in-picture effects, or any additional visual elements to a video.

The overlay filter can be applied with the **-filter_complex** option in FFmpeg, which is used for more complex filtering that involves multiple input streams (such as combining two videos or adding an image to a video).

The basic syntax for the overlay filter is as follows:

```
ffmpeg -i main_video.mp4 -i overlay.mp4 -filter_complex "overlay=x:y" output.mp4
```

Here, **main_video.mp4** is our primary video, and **overlay.mp4** is the video or image we want to overlay. The **x** and **y** values in the overlay filter specify the position of the overlay image/video on the main video.

As an example, let's say we want to add a company logo to the bottom-right corner of a video. First, we must prepare the files. In this case, we have the following:

- The main video file will be **video.mp4**
- The logo image will be **logo.png** (preferably with a transparent background)

Then, we will determine the logo's position. The logo's position will depend on the resolution of the main video. For example, if the video is 1920x1080 (full HD), and you want to place the logo 10 pixels from the bottom and right edges, the coordinates would be (x=1900, y=1060).

With this in mind, we will have to execute the following command:

```
ffmpeg -i video.mp4 -i logo.png -filter_complex "overlay=1900:1060" -codec:a copy output.mp4
```

In this command, we have the following:

- **-i video.mp4**: Specifies the main video file.
- **-i logo.png**: Specifies the overlay file (logo).
- **-filter_complex "overlay=1900:1060"**: Applies the overlay filter. The logo is positioned at (1900,1060), which is near the bottom-right corner.
- **-codec:a copy**: Copies the audio from the main video without re-encoding.
- **output.mp4**: The output file with the logo overlaid on the video.

Is this all we can do with the overlay filter? No, there's much more! For example, we can move this overlay dynamically.

Dynamic positioning with the overlay filter in FFmpeg

The overlay filter in FFmpeg not only allows static placement of images or videos over a main video but also offers dynamic positioning capabilities. This advanced feature enables the overlay to move across the screen or change its appearance over time, adding a dynamic element to your videos.

First, let's explore how to create the effect of moving an overlay across the screen. This technique is particularly effective for adding motion to logos, text, or other graphical elements.

Before we dive into the command, it's important to understand how FFmpeg processes expressions for movement. These expressions allow the position of the overlay to change frame by frame, creating the illusion of motion.

The command for moving an overlay is as follows:

```
ffmpeg -i main_video.mp4 -i logo.png -filter_complex "overlay=x='t*100':y=50" output.mp4
```

In this command, we have the following:

- **x='t*100'**: The horizontal position (x) of the overlay starts at 0 and increases by 100 pixels every second. The **t** variable represents the current time in seconds.
- **y=50**: The vertical position (y) is fixed at 50 pixels from the top of the frame.

We can play with these values to introduce different effects in our video overlay. For example, if we create a complete video editor, we could allow the users to move an element over the video and change its position during the video playback. Then, we could map those different positions to the seconds where we want it to be moved using FFmpeg. However, we won't be doing this as it would take another book entirely!

If you are curious about this, here is the documentation for the **overlay** parameter: <https://ffmpeg.org/ffmpeg-filters.html#overlay-1>.

Another feature we can use is fade-in and fade-out effects, which we can apply to our overlay. Let's see how it works.

Introducing the fade-in/out command

To achieve a fade-in/out effect, we combine the overlay filter with the fade filter. Let's break down the command to understand how it's structured:

```
ffmpeg -i main_video.mp4 -i logo.png -filter_complex "[1:v]fade=t=in:st=0:d=1,fade=t=out:st=3:d=1[
```

Let's understand how this command is configured:

- **[1:v]fade=t=in:st=0:d=1**: Applies a fade-in effect to the overlay, starting at 0 seconds and lasting for 1 second
- **fade=t=out:st=3:d=1[logo]**: Subsequently, a fade-out effect starts at 3 seconds and also lasts for 1 second
- **overlay=10:10**: The overlay is placed at the coordinates (10,10) on the main video

But there is more that we can do with FFmpeg, apart from using exposed filters. Let's see how we can use the **mobile-ffmpeg** library that we've already integrated into our project to improve the videos we are already recording.

Using mobile-ffmpeg to execute FFmpeg commands

With **mobile-ffmpeg** integrated, executing FFmpeg commands in Android becomes a streamlined process.

The library's **FFmpeg.execute()** method is the gateway to running FFmpeg commands. For instance, a command such as **-i input.mp4 -c:v libx264 output.mp4**, which converts an input video so that it uses the H.264 codec, is seamlessly executed within the Android environment. This function mirrors the command-line syntax of FFmpeg, maintaining familiarity for those accustomed to FFmpeg's command-line interface.

Here's how it would work:

```
val command = "-i input.mp4 -c:v libx264 output.mp4"
```

```
val returnCode = FFmpeg.execute(command)
```

In the previous code block, we are building a string with the **command** instruction and storing it in the **command** variable. Then, we are using the **FFmpeg.execute()** method to execute the command. Note that this execution will happen in the current thread, which could be undesirable performance-wise.

Managing performance and user experience is crucial in Android, especially for resource-intensive tasks such as video processing. **mobile-ffmpeg** accommodates this by offering asynchronous execution of commands. Utilizing **FFmpeg.executeAsync()** ensures that longer operations do not block the main thread, thus maintaining the application's responsiveness. This method becomes instrumental when handling complex transformations or filters, such as scaling a video.

Here's how we can use the **executeAsync** function:

```
FFmpeg.executeAsync(command) { executionId, returnCode ->
    when (returnCode) {
        Config.RETURN_CODE_SUCCESS -> {
            // Processing was successful
        }
        Config.RETURN_CODE_CANCEL -> {
            // Command execution was cancelled
        }
        else -> {
            // Command execution failed
        }
    }
}
```

In this example, the **executeAsync()** method is called with the **FFmpeg** command in string format. This command is what we intend FFmpeg to execute, such as converting a video file, applying filters, or any other media processing task supported by FFmpeg. The execution of this command occurs in a separate thread, preventing any blocking of the main UI thread of the application.

When the command has finished executing, a Lambda function is triggered. This function is structured to receive two parameters: **executionId** and **returnCode**. The **executionId** parameter is a unique identifier for this particular execution instance of the **FFmpeg** command and can be useful for tracking or managing this specific operation, especially if our application handles multiple FFmpeg processes concurrently.

The **returnCode** parameter is crucial as it indicates the outcome of the executed **FFmpeg** command. The different return codes and their implications are as follows:

- **Config.RETURN_CODE_SUCCESS:** This code signifies that the **FFmpeg** command was executed successfully without any errors. In the corresponding block of the **when** statement, you might want to implement functionality that deals with the successful completion of the media

processing task. This could include updating the user interface, processing or displaying the output file, or triggering subsequent application logic.

- **Config.RETURN_CODE_CANCEL:** This return code indicates that the execution of the **FFmpeg** command was canceled. This can occur if the execution is programmatically aborted or if certain external conditions pre-emptively stop the command. The handling code block for this return code could involve notifying the user of the cancellation, cleaning up resources, or setting the stage for a potential retry of the operation.
- **else:** This block catches all other cases, which generally suggests that an error occurred during the execution of the **FFmpeg** command. Here, error-handling strategies come into play, such as logging the error for diagnostic purposes, informing the user of the failure, or attempting to retry the operation under certain conditions.

To further refine the integration, **mobile-ffmpeg** allows us to handle progress and log outputs. This is essential for debugging and enhancing the user experience. Here's how it works:

```
FFmpeg.executeAsync(command, ExecuteCallback { executionId,
returnCode ->
    // Handle execution result
}, LogCallback { logMessage ->
    // Handle log message
}, StatisticsCallback { statistics ->
    // Handle progress updates
})
```

Here, **LogCallback** complements the execution callback that we described before. **FFmpeg** is known for its verbose logging, providing a wealth of information about the ongoing operation. The **logMessage** parameter in this callback gives you access to these logs, enabling you to handle them as per your application's needs. Whether it's displaying these logs for debugging purposes, analyzing them for detailed error reporting, or simply directing them to a file for record-keeping, this callback plays a pivotal role in understanding and managing the intricacies of **FFmpeg**'s operations.

Last but not least, **StatisticsCallback** opens the door to real-time monitoring of the **FFmpeg** process. This callback, through the **statistics** parameter, provides live data, such as the frame currently being processed, elapsed time, and bitrate, among others. Utilizing this data can significantly enhance the user experience, enabling you to implement dynamic features such as progress bars, estimated-time-to-completion indicators, or even detailed reports of the ongoing operation's status.

Now that we know how to execute our **FFmpeg** commands in Android, let's build something. We will start by adding a caption to the video.

Adding a caption to the video with FFmpeg

In this section, we will create all the components we'll need to add a caption to a video using FFmpeg. We'll start this new feature by creating a use case where the business logic of adding the caption to the video will be defined. We will call it **AddCaptionToVideoUseCase**, and its responsibility will be to add the caption to the video and return the new video file once it has been added.

This is how we can build **AddCaptionToVideoUseCase**:

```
class AddCaptionToVideoUseCase() {
    suspend fun addCaption(videoFile: File, captionText:
String): Result<File> = withContext(Dispatchers.IO) {
        val outputFile = File(
            videoFile.parent,
            "${videoFile.nameWithoutExtension}
            _captioned.mp4")
        val fontFilePath =
            "/system/fonts/Roboto-Regular.ttf"
        val ffmpegCommand = arrayOf(
            "-i", videoFile.absolutePath,
            "-vf", "drawtext=fontfile=$fontFilePath:
            text='$captionText':
            fontcolor=white:
            fontsize=24:x=(w-text_w)/2:
            y=(h-text_h)-10",
            "-c:a", "aac",
            "-b:a", "192k",
            outputFile.absolutePath
        )
        try {
            val executionId =
                FFmpeg.executeAsync(ffmpegCommand)
            { _, returnCode ->
                if (returnCode !=
                    Config.RETURN_CODE_SUCCESS) {
                    Result.failure<AddCaptionToVideoError>(
                        AddCaptionToVideoError)
                }
            }
            // Optionally handle the executionId, e.g., for
            // cancellation
            Result.success(outputFile)
        } catch (e: Exception) {
            Result.failure(e)
        }
    }
}

object AddCaptionToVideoError: Throwable("There was an
error adding the caption to the video") {
    private fun readResolve(): Any = AddCaptionToVideoError
}
```

In the preceding code, we start by creating a suspend function, **addCaption**, which is specifically designed to facilitate asynchronous execution via coroutines. As the action of adding a caption involves intensive tasks such as video processing, we should avoid executing this kind of logic in the main thread to prevent any lag or unresponsiveness in the applica-

tion. The function takes two parameters: a **File** object representing the video file and a **String** containing the caption text to be added.

Inside the **addCaption** function, the execution context is switched to the I/O dispatcher. This is done to optimize for I/O operations, ensuring that the file processing workload is handled appropriately without straining the main thread. The function proceeds to create an **outputFile** object. This object represents the new video file that will be generated post-captioning.

The next segment in the function involves constructing a command string for FFmpeg. This command is carefully crafted to utilize FFmpeg's **drawtext** filter, enabling the provided caption text to be overlaid on the video. Let's analyze the command that we used in the previous code block:

```
val command = "-i ${videoFile.absolutePath} -vf drawtext=text='${captionText':fontcolor=white:fontsize=24:x=(w-text_w)/2:y=(h-text_h)/2:codec:a=acc:-b:a=192k:${outputFile.absolutePath}"
```

Let's break this command down:

- **-i \${videoFile.absolutePath}**: This part of the command specifies the input file for FFmpeg to process. The **-i** flag is used for input files in FFmpeg and **\${videoFile.absolutePath}** dynamically inserts the absolute path of the video file you're processing.
- **-vf drawtext=text='\${captionText':...**: The **-vf** (video filter) flag is used to apply filters to the video. Here, the **drawtext** filter is used to add text to the video.
- **text='\${captionText}'**: This specifies the text to be drawn. Here, **\$captionText** is the variable holding the caption text, which is dynamically inserted into the command.
- **fontcolor=white**: Sets the font color of the text to white.
- **fontsize=24**: Defines the size of the font used for the text.
- **x=(w-text_w)/2**: This sets the horizontal position of the text. Here, **w** represents the width of the video, and **text_w** is the width of the text. By setting **x** to **(w-text_w)/2**, the text is horizontally centered.
- **y=(h-text_h)/2**: Similarly, this sets the vertical position of the text. Here, **h** is the height of the video, and **text_h** is the height of the text. This formula vertically centers the text within the video.
- **-codec:a acc**: This part of the command instructs FFmpeg to use **acc** as the codec for audio streaming.
- **-b:a=192k**: This part of the command sets the bitrate to 192k.
- **\${outputFile.absolutePath}**: The last part of the command specifies the output file's path, where the processed video (with the caption added) will be saved.

Executing this FFmpeg command is handled asynchronously with **FFmpeg.executeAsync()**. This method is pivotal for running the command in a non-blocking manner and is accompanied by a Lambda function for handling the execution result. The Lambda function evaluates **returnCode** from the FFmpeg execution. In the case of a non-successful execution (indicated by any return code other than **RETURN_CODE_SUCCESS**), the function constructs **Result.failure**, wrap-

ping a custom **AddCaptionToVideoError** object. This custom error object, defined as a singleton, provides a specific error message indicating an issue with the captioning process.

On the flip side, successful command execution results in **Result.success**, passing along **outputFile**. This bifurcation in handling success and failure scenarios ensures robust error management and clear feedback regarding the outcome of the captioning process.

Now, we can use **AddCaptionToVideoUseCase** in **StoryEditorViewModel**:

```
class StoryEditorViewModel(
    private val saveCaptureUseCase: SaveCaptureUseCase,
    private val addCaptionToVideoUseCase:
        AddCaptionToVideoUseCase
): ViewModel() {
    // Other variables we defined for the photo feature
    var videoFile: File? = null
    // Other code we already added for the photo feature
    fun addCaptionToVideo(captionText: String) {
        videoFile?.let { file ->
            viewModelScope.launch {
                val result =
                    addCaptionToVideoUseCase.addCaption(
                        file, captionText)
                // Handle the result of the captioning
                process
            }
        }
    }
}
```

We start by injecting **AddCaptionToVideoUseCase** into **StoryEditorViewModel** using its constructor. Then, we declare a **videoFile** variable in **ViewModel**, which holds the video we're working with – it's nullable because there might be times when we don't have a video to display or edit. In **videoFile**, we should have stored the view we have already recorded.

Next, the core function in this **ViewModel** is **addCaptionToVideo**. This function takes the caption text as input and uses the video file we have. First, it checks if **videoFile** isn't **null**. If we have a video, it proceeds; if not, nothing happens.

Inside **addCaptionToVideo**, by launching a coroutine within **viewModelScope**, we ensure that our caption-adding process doesn't freeze the UI. This is crucial for maintaining a smooth user experience.

The **addCaption** method of our use case is then called with the video file and caption text. Whatever comes back from this operation – success or failure – is stored in the result.

The **// Handle the result of the captioning process** comment is where you'd put our code to update the UI based on the result. This could mean displaying the captioned video, showing an error message, or what-

ever else makes sense for our app. For simplicity, we won't be adding it here just yet, but we will learn more about video playback in the last three chapters of this book when we create a Netflix-esque app.

But we can still test the effect in our video. We just have to look at the internal app files using Device Explorer in Android Studio. There, we'll see two files – one of the original video, and the other a modified one with the `_captioned` suffix:

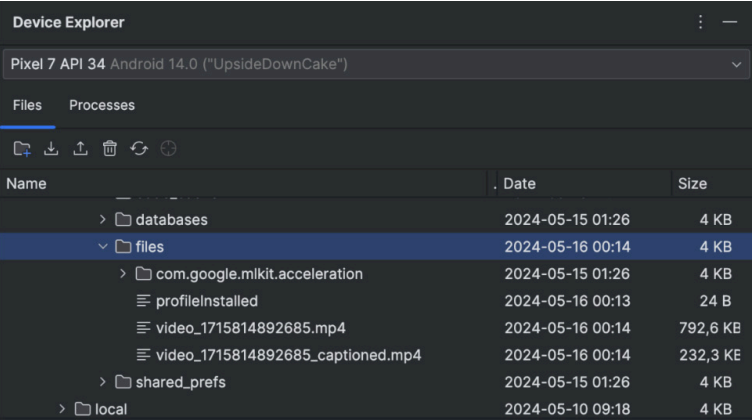


Figure 6.3: Device Explorer with video files in Android Studio

If we download the captioned file video and play it, we should see that a caption has been added to the video:



Figure 6.4: A video with caption text stating, “This is the caption text”

Now that we know how to apply a caption to our video, let’s see how we can apply a filter.

Adding a filter to a video with FFmpeg

In this section, we will learn how to add a filter to our video. A popular filter that is visually impactful is the “vignette” effect – this effect typically darkens the edges of the frame, drawing the viewer’s attention toward

the center of the image or video, and can add a dramatic or cinematic quality to the footage. FFmpeg has the capability to apply this artistic filter to videos, so let's try it out!

As we did with the caption, we will start by creating the use case:

AddVignetteEffectUseCase. The primary role of **AddVignetteEffectUseCase** is to execute the business logic for applying the vignette effect to a given video file by using **mobile-ffmpeg**. We will use a specific **FFmpeg** command, as follows:

```
class AddVignetteEffectUseCase() {
    suspend fun addVignetteEffect(videoFile: File):
    Result<File> = withContext(Dispatchers.IO) {
        val outputFile = File(videoFile.parent,
            "${videoFile.nameWithoutExtension}
            _vignetted.mp4")
        val command = "-i ${videoFile.absolutePath} -vf
            vignette=angle=PI/4 ${outputFile.absolutePath}"
        try {
            val executionId = FFmpeg.executeAsync(command)
            { _, returnCode ->
                if (returnCode !=
                    Config.RETURN_CODE_SUCCESS) {
                    Result.failure<AddVignetteEffectError>(
                        AddVignetteEffectError()
                    )
                }
            }
            // Optionally handle the executionId, e.g., for
            // cancellation
            Result.success(outputFile)
        } catch (e: Exception) {
            Result.failure(e)
        }
    }
}

object AddVignetteEffectError : Throwable("There was an
error adding the vignette effect to the video") {
    private fun readResolve(): Any = AddVignetteEffectError
}
```

Let's walk through the code in **AddVignetteEffectUseCase**. Here, **addVignetteEffect** is a suspend function, meaning it's designed to run asynchronously with Kotlin's coroutines. In this function, we take the video file that needs the vignette effect and start by defining where to save the processed video. We keep the original video intact and create a new file for the output. The output's filename keeps the original name but with **_vignetted** added to it so that it's easy to track.

Next up, we build our FFmpeg command. This command tells FFmpeg to apply the vignette effect. Let's see how this command (already present in the previous code block) works in detail:

```
val command = "-i ${videoFile.absolutePath} -vf vignette=angle=PI/4 ${outputFile.absolutePath}"
```

This command is composed of the following segments:

- **-i `${videoFile.absolutePath}`**: This part of the command specifies the input file for FFmpeg to process. The **-i** flag is used for input files in FFmpeg and **`${videoFile.absolutePath}`** dynamically inserts the absolute path of the video file you want to process. In simple terms, it tells FFmpeg, “Here’s the video I want you to work on.”
- **-vf `vignette=angle=PI/4`**: This segment is where the vignette effect is applied.
- **-vf** stands for video filters and is a powerful feature in FFmpeg that allows you to apply various transformations or effects to your video.
- **`vignette=angle=PI/4`**: This is the specific filter and setting for the vignette effect. The vignette filter in FFmpeg is used to apply the vignette effect, which typically darkens the edges of the video to focus attention on the center. The **`angle=PI/4`** part is a parameter of the vignette filter that controls the angle of the effect. This specific setting, **`PI/4`**, is chosen to give a visually pleasing vignette. It’s a bit of a creative choice, balancing subtlety and impact.
- **`${outputFile.absolutePath}`**: The last part of the command specifies where to save the processed video. It takes the path where you want the new video (with the vignette effect applied) to be saved. By placing it here in the command, you’re telling FFmpeg, “Once you’re done adding the effect, save the new video here.”

When it comes to running this command, we use **`FFmpeg.executeAsync`**. This method is great because it runs our command without blocking the app. The method also has a way to check if everything went as planned. If the command runs successfully, we return the path of our new vignette video. But if something goes wrong, we catch it and return an error. Here, **`AddVignetteEffectError`** is a custom error message we throw if the FFmpeg command doesn’t execute properly. It’s a simple way to know exactly what went wrong when we add our vignette effect. And with this, **`AddVignetteUseCase`** is ready.

Now, we can integrate this use case into **`StoryEditorViewModel`**:

```
class StoryEditorViewModel(
    private val saveCaptureUseCase: SaveCaptureUseCase,
    private val addCaptionToVideoUseCase:
        AddCaptionToVideoUseCase,
    private val addVignetteEffectUseCase:
        AddVignetteEffectUseCase
): ViewModel() {
    ...
    var videoFile: File? = null
    ...
    fun addVignetteFilterToVideo() {
        videoFile?.let { file ->
            viewModelScope.launch {
                val result =
                    addVignetteEffectUseCase
                        .addVignetteEffect(file)
                // Handle the result of the filter process
            }
        }
    }
}
```

```
    }  
}
```

Here, **StoryEditorViewModel** is structured to receive **AddVignetteEffectUseCase** as a dependency.

Within this ViewModel, we maintain a **videoFile** property, which holds a reference to the video file that the vignette effect will be applied to. The nullable nature of this property allows for scenarios where a video file may not be immediately available.

The function to execute this functionality is **addVignetteEffectToVideo**. When invoked, this function checks whether **videoFile** is not null, ensuring that there is a valid file to process. If a video file is available, the function proceeds to launch a coroutine within **viewModelScope**.

Inside the coroutine, the **addVignetteEffectUseCase.addVignetteEffect** method is called with the video file as its argument. This is where the vignette effect is applied to the video. The result of this operation is captured in a variable named **result**. This result could indicate either a successful application of the effect or a failure due to some error.

The commented section within the function, **// Handle the result of the vignette effect process**, is where we would typically handle the outcome of the operation. Depending on whether the vignette effect was successfully applied or not, this section could include code for updating the UI to display the processed video or handling any errors that might have occurred during the process.

As we mentioned when we discussed adding captions, we haven't implemented video playback yet, but we can still test the effect in our video. Just like back in *Figure 6.3*, we can see two files, but this time one of them has a **_vignetted** suffix to indicate it has been modified:

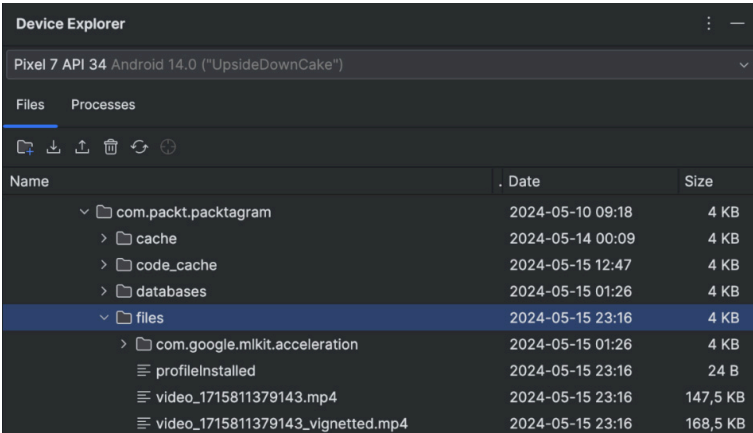


Figure 6.5: Device Explorer in Android Studio

We can download and reproduce both videos to check and test the filter:



Figure 6.6: Video without (left) and with (right) the vignette filter effect applied

Now that we know how to integrate FFmpeg and use its commands to edit the user's videos, it is time to upload those videos so that they can be shared between their contacts.

Uploading the video

Now that our video is ready, it's time to upload it to any service, from where it can be shared with the user contacts. We're going to use Firebase Storage for this (to learn how to set up Firebase Storage, please refer to [Chapter 3](#)).

We'll start by creating a data source that will be responsible for uploading the video to Firebase Storage. We will call it **VideoStorageDataSource**:

```
class VideoStorageDataSource {
    fun uploadVideo(videoFile: File, onSuccess: (String) ->
Unit, onError: (Exception) -> Unit) {
        val storageReference =
            FirebaseStorage.getInstance().reference
        val videoRef = storageReference.child(
            "videos/${videoFile.name}")
        val uploadTask =
            videoRef.putFile(Uri.fromFile(videoFile))
        uploadTask.addOnSuccessListener {
            videoRef.downloadUrl.addOnSuccessListener { uri ->
                onSuccess(uri.toString())
            }
        }.addOnFailureListener { exception ->
            onError(exception)
        }
    }
}
```

```

    }
  }
}

```

Inside the **uploadVideo** function, we start indicating that we'll execute the logic in the I/O dispatcher.

Then, the heart of the function is where we use Firebase Storage. First, we obtain the reference of the storage using

FirestoreStorage.getInstance().reference, after which we set up a reference to where we want our video to be stored in Firebase using **storageReference.child("videos/\${videoFile.name}")**.

Next, we start the upload itself. The **putFile** method is used to upload the video file. This is where **await()** comes into play. This **await()** is a suspending function that patiently waits for the upload to complete without blocking the thread. It's part of Kotlin's coroutines magic and is a game-changer for async operations.

Once the upload is done, we need to grab the URL of our video. So, we call **downloadUrl.await()**. Just like with the upload, **await()** suspends the operation until Firebase gives us the video's URL.

We've also got our error handling covered. The upload and URL retrieval process is wrapped in a **try-catch** block. If anything goes sideways during the upload or while fetching the URL, we catch the exception and wrap it up in **Result.failure(e)**. On the other hand, if all goes well, we return **Result.success(downloadUrl.toString())**, handing over the URL of the newly uploaded video.

Next, we will implement the repository that will be responsible for managing and connecting the data sources to the domain layer. We will call its interface **VideoRepository** and the implementation

VideoRepositoryImpl:

```

interface VideoRepository {
    suspend fun uploadVideo(videoFile: File):
        Result<String>
}
class VideoRepositoryImpl(private val
    videoStorageDataSource: VideoStorageDataSource) :
    VideoRepository {
    override suspend fun uploadVideo(videoFile: File):
        Result<String> {
        return try {
            var uploadResult: Result<String> =
                Result.failure(RuntimeException("Upload
                    failed"))
            firebaseStorageDataSource.uploadVideo(
                videoFile, { url ->
                    uploadResult = Result.success(url)
                }, { exception ->
                    uploadResult = Result.failure(exception)
                })
            uploadResult
        }
    }
}

```

```

        } catch (e: Exception) {
            Result.failure(e)
        }
    }
}

```

First up, we have our **VideoRepository** interface. This is a straightforward Kotlin interface with one key function: **uploadVideo**.

Next, we have the **VideoRepositoryImpl** class, which implements the **VideoRepository** interface. This class is where the action happens. It's initialized with an instance of **VideoStorageDataSource**.

Then, the **uploadVideo** function follows a **try-catch** pattern for robust error handling. Initially, it sets up a default **uploadResult** as a failure. This is a cautious approach, assuming things might go wrong, and we'll update this only if the upload succeeds.

Then, we call **uploadVideo** on **videoStorageDataSource**, passing the video file along with two Lambda functions for handling success and failure. If the upload is successful, the success Lambda updates **uploadResult** with the URL of the uploaded video. If there's a failure, the failure Lambda updates **uploadResult** with the encountered exception.

Finally, we return **uploadResult**. If all goes well, we'll see the URL of the uploaded video. If not, we'll see the error that occurred during the process. The **try-catch** block ensures that if there's an unexpected exception anywhere in this process, we catch it and return it as a failure.

Now, it's time for us to implement **UploadVideoUseCase**:

```

class UploadVideoUseCase(private val videoRepository:
VideoRepository) {
    suspend fun uploadVideo(videoFile: File):
Result<String> {
        return videoRepository.uploadVideo(videoFile)
    }
}

```

Here, we are injecting **VideoRepository**. In the **uploadVideo** function, we call **videoRepository** and pass **videoFile** as a parameter.

Finally, we will include **UploadVideoUseCase** in **StoryEditorViewModel** and use it from there:

```

class StoryEditorViewModel(
    private val saveCaptureUseCase: SaveCaptureUseCase,
    private val addCaptionToVideoUseCase:
        AddCaptionToVideoUseCase,
    private val addVignetteEffectUseCase:
        AddVignetteEffectUseCase,
    private val uploadVideoUseCase: UploadVideoUseCase
) : ViewModel() {
    ...
}

```

```
fun uploadVideo(videoFile: File) {  
    viewModelScope.launch {  
        val result =  
            uploadVideoUseCase.uploadVideo(videoFile)  
        // Handle the result of the upload process  
    }  
}
```

In **StoryEditorViewModel**, we add a function called **uploadVideo** that takes the video file and uses **uploadVideoUseCase** to upload it. The operation is performed within a coroutine to ensure it doesn't block the UI thread.

The **// Handle the result of the upload process** comment is where we would implement the logic based on the outcome of the upload. If the upload is successful, we might update the UI to show that the video has been uploaded or display the video URL. In case of failure, we would handle the error, perhaps by showing an error message to the user.

And with this change, we are ready to upload the video from our **ViewModel**. By doing this, we have completed this chapter, as well as our work on Packtagram!

Summary

Wrapping up this chapter, you've significantly leveled up your Packtagram app's video capabilities.

Starting with CameraX, we expanded its use from snapping photos to capturing high-quality videos, but this was just the beginning. Then, we dived into FFmpeg, an incredibly versatile tool for video editing. Here, you learned how to add a creative touch to videos, be it through captions that tell a story or filters that change the entire look and feel.

But what's a great video if it can't be shared? We tackled that too by integrating Firebase Storage for seamless video uploads. This means your app is now adept at handling large files smoothly, ensuring users enjoy a hiccup-free experience.

With this chapter, we have finished our work on Packtagram. Now, it's time to learn about the project that will be implemented in the last three chapters: a video playback app so that you can view your favorite series and films!